

Ingegneria del Software – Canale 2

Prof. Bernardo Breve

Esercitazione

Immagina di lavorare su un sistema software che gestisce una fabbrica di pelati. La fabbrica si occupa della produzione, del confezionamento e della distribuzione di pelati in scatola. Le operazioni principali riguardano la gestione delle materie prime (pomodori, spezie, ecc.), il processo di produzione, il confezionamento e l'invio degli ordini ai distributori.

```
public class GestoreFabbricaPelati {
    private Connection conn;
    public Map<Integer, MateriaPrima> cacheMateriePrime = new HashMap<>();
    public Map<Integer, Ordine> cacheOrdini = new HashMap<>();

    public GestoreFabbricaPelati() {
        try {
            conn =
DriverManager.getConnection("jdbc:postgresql://fabbrica.pelati.com:5432/fabbrica
_pelati", "user", "pw");
        } catch (SQLException e) { e.printStackTrace(); }
    }

    public void aggiungiMateriaPrima(MateriaPrima m) throws SQLException {
        String sql = "INSERT INTO materie_prime VALUES (?, ?, ?)";
        PreparedStatement ps = conn.prepareStatement(sql);
        ps.setInt(1, m.getId());
        ps.setString(2, m.getNome());
        ps.setInt(3, m.getQuantita());
        ps.executeUpdate();
        inviaEmailDistributore("distributore@pelati.it", "Nuova materia prima
aggiunta");
    }

    public void avviaProduzione(int idOrdine) throws SQLException {
        Ordine ordine = cacheOrdini.get(idOrdine);
        // Logica di produzione...
        System.out.println("Produzione per ordine " + idOrdine + " avviata.");
        inviaEmailDistributore("distributore@pelati.it", "Produzione ordine " +
idOrdine + " completata");
    }

    public void inviaOrdine(int idOrdine) throws SQLException {
        Ordine ordine = cacheOrdini.get(idOrdine);
        // Logica di spedizione...
        System.out.println("Ordine " + idOrdine + " spedito al distributore.");
        inviaEmailDistributore("distributore@pelati.it", "Ordine " + idOrdine +
" spedito");
    }

    private void inviaEmailDistributore(String to, String body) { /* ... */ }
}
```

- Individuare e descrivere i problemi presenti nel codice che determinano bassa coesione e alto accoppiamento.
- Proporre una nuova decomposizione del sistema in sottosistemi o classi, che migliori coesione e riduca accoppiamento.